Executive Summary — SMS Spam Detection

Page 1 — Problem, solution, and evidence

The problem

SMS users may receive unwanted advertisements, fake prize notices, and messages designed to trick recipients into sharing personal information. A simple classifier can help flag messages that resemble spam so users can review them more carefully.

Proposed solution

SMS Spam Detector is an end-to-end machine-learning prototype. It accepts the text of an SMS and predicts whether it resembles spam or a normal message. A Streamlit interface displays the predicted class and an estimated spam score.

Data and model

The repository includes a small illustrative CSV with message and label fields. Labels are spam and ham. The model uses TF-IDF text features and Logistic Regression. The data-cleaning step removes missing values, blank messages, invalid labels, and duplicates. The training script uses a held-out stratified test split.

Evaluation plan

The pipeline reports accuracy, spam precision, spam recall, F1-score, and a confusion matrix. Run python train.py to generate actual values under reports/. The included sample is small and intended only to demonstrate the code workflow. Results should not be presented as evidence of real-world performance.

Page 2 — Value, delivery, and next steps

Intended value

The prototype demonstrates how text classification may help users identify messages that deserve extra caution. It is an assistive tool only; it should not automatically delete messages or determine whether a message is safe.

Deliverables

• Reproducible sample dataset and training pipeline.

• TF-IDF and Logistic Regression model.

• Evaluation metrics and confusion matrix.

• Streamlit web interface for testing messages.

• Project report and five-slide presentation.

Risks and safeguards

False positives can flag legitimate messages, while false negatives can miss harmful messages. The sample may not represent modern scams, different languages, regional wording, or changing spam tactics. Avoid entering sensitive messages into an untrusted deployment. Do not automatically block or delete messages based only on the prediction.

Implementation plan

1. Run the prototype on the included sample.

2. Replace the sample with a larger, documented dataset whose terms permit use.

3. Compare Logistic Regression with Naive Bayes and Linear SVM.

4. Tune the decision threshold after considering the cost of false positives and false negatives.

5. Add privacy safeguards, language coverage, feedback, and monitoring.

6. Validate performance on data separate from training before any real deployment.

Conclusion

This capstone demonstrates the main machine-learning workflow from problem framing and data preparation to model evaluation, interface, and presentation. The current version is an educational prototype, not a production-ready spam filter.